

761. Special Binary String

Solved 

Hard

 Topics

 Companies

 Hint

Special binary strings are binary strings with the following two properties:

- The number of `0`'s is equal to the number of `1`'s.
- Every prefix of the binary string has at least as many `1`'s as `0`'s.

You are given a **special binary** string `s`.

A move consists of choosing two consecutive, non-empty, special substrings of `s`, and swapping them. Two strings are consecutive if the last character of the first string is exactly one index before the first character of the second string.

Return *the lexicographically largest resulting string possible after applying the mentioned operations on the string*.

Example 1:

Input: `s = "11011000"`

Output: `"11100100"`

Explanation: The strings `"10"` [occurring at `s[1]`] and `"1100"` [at `s[3]`] are swapped.

This is the lexicographically largest string possible after some number of swaps.

Example 2:

Input: `s = "10"`

Output: `"10"`

Constraints:

- `1 <= s.length <= 50`
- `s[i]` is either `'0'` or `'1'`.
- `s` is a special binary string.

Python:

class Solution:

def makeLargestSpecial(self, s: str) -> str:

count = 0

i = 0

res = []

```

for j in range(len(s)):
    # Track balance: +1 for '1', -1 for '0'
    count += 1 if s[j] == '1' else -1

    # Found a balanced chunk when count returns to 0
    if count == 0:
        # Recursively maximize inner part, wrap with 1...0
        res.append('1' + self.makeLargestSpecial(s[i + 1:j]) + '0')
        i = j + 1 # Move to next potential chunk

# Sort chunks in descending order for largest arrangement
res.sort(reverse=True)
return ''.join(res)

```

JavaScript:

```

const cache = new Map();

const makeLargestSpecial = s => {
    if (cache.has(s)) return cache.get(s);

    let streak = 0, i = 0;
    const arr = [];

    for (let j = 0; j < s.length; j++) {
        streak += ((s[j] === '1') << 1) - 1;

        if (!streak) {
            arr.push(`1${makeLargestSpecial(s.slice(i + 1, j))}0`);
            i = j + 1;
        }
    }

    const res = arr.sort().reverse().join``;
    cache.set(s, res);

    return res;
};

```

Java:

```

class Solution {
    public String makeLargestSpecial(String s) {
        int count = 0, i = 0;
        List<String> res = new ArrayList<>();
    }
}

```

```

for (int j = 0; j < s.length(); j++) {
    // Track balance: +1 for '1', -1 for '0'
    if (s.charAt(j) == '1') count++;
    else count--;

    // Found a balanced chunk when count returns to 0
    if (count == 0) {
        // Recursively maximize inner part, wrap with 1...0
        res.add('1' + makeLargestSpecial(s.substring(i + 1, j)) + '0');
        i = j + 1; // Move to next potential chunk
    }
}

// Sort chunks in descending order for largest arrangement
Collections.sort(res, Collections.reverseOrder());
return String.join("", res);
}
}

```